

TRƯỜNG ĐẠI HỌC BÌNH DƯƠNG
KHOA CÔNG NGHỆ THÔNG TIN, ROBOT
VÀ TRÍ TUỆ NHÂN TẠO



---oOo---

TIỂU LUẬN CUỐI KỲ
MÔN: CHUYÊN ĐỀ 1

ĐỀ TÀI: QUẢN LÝ KHÓA HỌC

Giảng viên hướng dẫn: Th.S Dương Anh Tuấn

Sinh viên thực hiện: Huỳnh Minh Chiến - 22050025

Bình Dương, tháng 7/2026

LỜI CẢM ƠN

Trước hết, tôi xin bày tỏ lòng biết ơn đến quý thầy cô Trường Đại học Bình Dương đã giảng dạy, truyền đạt kiến thức và tạo điều kiện để tôi vận dụng kiến thức công nghệ thông tin vào việc xây dựng một sản phẩm phần mềm có tính thực tiễn.

Tôi xin trân trọng cảm ơn quý thầy cô giảng dạy và hướng dẫn học phần Chuyên đề 1. Những góp ý về cách xác định bài toán, phân tích yêu cầu, thiết kế kiến trúc và triển khai hệ thống trên nền tảng Cloud là cơ sở để tôi hoàn thiện đề tài.

Tôi cũng xin cảm ơn gia đình, bạn bè đã động viên trong quá trình thực hiện. Mặc dù đã nỗ lực, báo cáo vẫn có thể còn thiếu sót do giới hạn thời gian và kinh nghiệm thực tế. Tôi mong nhận được những nhận xét của quý thầy cô để tiếp tục hoàn thiện sản phẩm.

Em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Chữ ký giảng viên

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Chữ ký giảng viên

LỜI CAM ĐOAN

Tôi cam đoan báo cáo “Xây dựng Hệ thống quản lý khóa học” là kết quả của quá trình học tập, phân tích, thiết kế, lập trình và triển khai của bản thân trong học phần Chuyên đề 1. Các nội dung tham khảo được liệt kê trong phần Tài liệu tham khảo; các số liệu minh chứng triển khai được lấy từ môi trường Cloud của dự án.

Tôi chịu trách nhiệm về tính trung thực của nội dung báo cáo và cam kết không sao chép trái phép sản phẩm của người khác. Các tài khoản, dữ liệu bệnh nhân, chuyên khoa và thuốc trong hệ thống là dữ liệu mô phỏng phục vụ mục đích học tập, kiểm thử và trình diễn.

Sinh viên thực hiện

TÓM TẮT

"Báo cáo trình bày quá trình xây dựng Hệ thống Web Quản lý Khóa học Trực tuyến (Course Management System) - một hệ thống web full-stack hỗ trợ quá trình học tập và quản lý các khóa học cơ bản. Hệ thống giải quyết nhu cầu tương tác giữa học viên và ban quản trị: học viên có thể đăng ký, đăng nhập, tìm kiếm khóa học, ghi danh và tham gia các bài học; trong khi đó, quản trị viên có quyền quản lý danh mục khóa học, nội dung bài học và quản lý tài khoản người dùng.

Kiến trúc hệ thống được thiết kế theo mô hình Client-Server thông qua các RESTful API. Tầng giao diện (Frontend) được phát triển bằng HTML, CSS và JavaScript thuần (Vanilla JS); tầng nghiệp vụ (Backend) sử dụng Node.js kết hợp framework Express.js đảm nhận việc xử lý logic, kiểm duyệt dữ liệu đầu vào và phân quyền; tầng dữ liệu sử dụng MongoDB kết hợp cùng Mongoose ORM. Hệ thống áp dụng cơ chế xác thực bảo mật bằng JWT (JSON Web Token), cơ chế CORS và phân quyền dựa trên vai trò (RBAC) để bảo vệ các API theo 02 vai trò chính: USER (Học viên) và ADMIN (Quản trị viên).

Sản phẩm hỗ trợ triển khai thực tế trên Cloud với Vercel cho frontend (thông qua cấu hình vercel.json), và backend Node.js kết nối với cơ sở dữ liệu MongoDB. Kết quả kiểm thử cho thấy các RESTful API hoạt động ổn định, dữ liệu mẫu (courses, lessons, users) được khởi tạo (seed) thành công, các luồng nghiệp vụ như đăng ký, đăng nhập, phân quyền, hiển thị và tương tác với khóa học đều diễn ra trơn tru. Sản phẩm đáp ứng đầy đủ yêu cầu của một ứng dụng Web Full-Stack hiện đại và có khả năng vận hành thực tế."

MỤC LỤC

LỜI CẢM ƠN	2
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	3
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN	4
LỜI CAM ĐOAN	5
TÓM TẮT	6
DANH MỤC HÌNH	10
DANH MỤC BẢNG	11
MỞ ĐẦU	12
1. Lý do chọn đề tài	12
2. Mục tiêu nghiên cứu và xây dựng	12
3. Đối tượng và phạm vi nghiên cứu	12
4. Phương pháp thực hiện	12
5. Tóm tắt nội dung báo cáo	13
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT	14
1.1. Bối cảnh và mô tả bài toán	14
1.2. Mục tiêu hệ thống	14
1.3. Đối tượng sử dụng	15
1.4. Phạm vi thực hiện	16
1.5. Khảo sát yêu cầu	16
1.5.1. Yêu cầu chức năng	16
1.5.2. Yêu cầu phi chức năng	17
1.6. Cơ sở lý thuyết	17
1.6.1. Kiến trúc 03 tầng	17
1.6.2. RESTful API	18

1.6.3. Authentication và Authorization	18
1.6.4. Thiết kế cơ sở dữ liệu quan hệ	18
1.6.5. Cloud Computing	18
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	19
2.1. Phân tích tác nhân và Use Case	19
2.2. Ma trận phân quyền	20
2.3. Quy trình nghiệp vụ chính	20
2.4. Sơ đồ luồng dữ liệu DFD mức ngữ cảnh	21
2.5. Sơ đồ hoạt động quy trình ghi danh	22
2.6. Sơ đồ tuần tự quy trình ghi danh	23
2.7. Quy tắc nghiệp vụ cốt lõi	25
2.8. Thiết kế kiến trúc tổng thể	25
2.9. Thiết kế frontend	26
2.10. Thiết kế backend	27
2.11. Thiết kế cơ sở dữ liệu	27
2.12. Data Dictionary rút gọn	28
2.13. Thiết kế giao diện và luồng thao tác	29
CHƯƠNG 3. XÂY DỰNG VÀ ĐÁNH GIÁ HỆ THỐNG	30
3.1. Xây dựng Frontend	30
3.2. Xây dựng Backend	31
3.3. Xây dựng Database	31
3.4. Authentication và Authorization	32
3.5. Các API endpoint tiêu biểu	32
3.6. Minh họa giao diện quản trị	34
3.7. Kiểm thử hệ thống	34
3.8. Đánh giá kết quả	35
3.8.1. Ưu điểm	35

3.8.2. Hạn chế	36
3.8.3. Hướng phát triển	36
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG TRÊN CLOUD	37
4.1. Kiến trúc triển khai	37
4.2. Cấu hình môi trường triển khai	38
4.3. Các bước triển khai	38
4.4. Minh chứng cấu hình và vận hành	39
4.5. URL truy cập và kiểm thử	40
4.6. Bảo mật cấu hình triển khai	40
4.7. Hạn chế của môi trường miễn phí	41
4.8. Source code	41
KẾT LUẬN	42
TÀI LIỆU THAM KHẢO	43

DANH MỤC HÌNH

Hình 2.1. Use Case tổng quát	19
Hình 2.2. Luồng trạng thái ghi danh và tham gia khóa học cầu học viên	21
Hình 2.3. Sơ đồ luồng dữ liệu mức ngữ cảnh	22
Hình 2.4. Sơ đồ hoạt động quy trình ghi danh	23
Hình 2.5. Sơ đồ tuần tự quy trình ghi danh	24
Hình 2.6. Kiến trúc 03 tầng	26
Hình 2.7. ERD rút gọn	28
Hình 3.1. Giao diện tổng quan	30
Hình 3.2. Giao diện chi tiết khóa học	31
Hình 3.3. Luồng đăng nhập, phân quyền RBAC	32
Hình 3.4. Giao diện tổng quan quản trị	34
Hình 4.1. Kiến trúc triển khai Cloud thực tế	37
Hình 4.2. Cấu hình Build Command, Start Command và Free instance trên Render	39
Hình 4.3. Cấu hình Frontend trên Vercel	40

DANH MỤC BẢNG

Bảng 1.2.Mục tiêu hệ thống.....	15
Bảng 2.Đối tượng sử dụng.....	15
Bảng 1.5.1. Yêu cầu chức năng.....	16
Bảng 1.5.2.Phi chức năng.....	17
Bảng 2.2.Phân quyền.....	20
Bảng 2.7.Quy tắc nghiệp vụ cốt lõi.....	25
Bảng 2.12 Data Dictionary.....	29
Bảng 3.5 API tiêu biểu.....	33
Bảng3.7. Kiểm thử.....	35
Bảng 4.2 cấu hình môi trường triển khai.....	38
Bảng 4.5 URL truy cập.....	40

MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh nhu cầu học tập và đào tạo trực tuyến ngày càng tăng cao, việc quản lý nội dung khóa học và tiến độ học tập đóng vai trò rất quan trọng. Khi thông tin bài giảng, danh sách ghi danh và kết quả học tập bị phân tán hoặc quản lý thủ công, học viên sẽ gặp khó khăn trong việc theo dõi lộ trình; giảng viên và ban quản trị cũng mất nhiều thời gian để cập nhật nội dung, kiểm soát tài khoản và giao dịch học phí. Vì vậy, đề tài "Xây dựng hệ thống Web quản lý khóa học trực tuyến" được thực hiện nhằm cung cấp một nền tảng số hóa tập trung, giải quyết trọn vẹn quy trình kết nối giữa người học và nhà cung cấp khóa học.

2. Mục tiêu nghiên cứu và xây dựng

Mục tiêu của đề tài là xây dựng một ứng dụng web full-stack vận hành thực tế trên Cloud, được thiết kế theo mô hình Client - Server tách biệt rõ ràng tầng giao diện và tầng nghiệp vụ. Hệ thống hướng tới hỗ trợ toàn diện các tính năng: đăng ký/đăng nhập, quản lý hồ sơ cá nhân và số dư tài khoản (balance), tìm kiếm và ghi danh khóa học (enrollment), tham gia bài học (lesson), đánh giá khóa học (review) và quản trị tập trung toàn bộ dữ liệu danh mục theo quyền hạn của từng vai trò.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là quy trình phân phối nội dung giáo dục và quản lý tiến độ học tập trực tuyến. Phạm vi hệ thống tập trung phục vụ 02 vai trò cốt lõi: User (Học viên) và Admin (Quản trị viên). Trong giới hạn của đề tài, hệ thống tập trung mô phỏng thanh toán học phí qua hệ thống ví/số dư nội bộ (transactions) thay vì tích hợp các cổng thanh toán trực tuyến thực tế (như VNPay, MoMo), và chưa bao gồm các tính năng phức tạp như phòng học trực tuyến (livestreaming) hay thi trắc nghiệm tính điểm tự động.

4. Phương pháp thực hiện

Đề tài được thực hiện tuần tự theo các bước: khảo sát quy trình vận hành của các nền tảng e-learning; phân tích yêu cầu chức năng và phi chức năng; thiết kế hệ thống qua các sơ đồ Use Case, DFD, Activity, Sequence và ERD. Ở giai đoạn thực thi, hệ thống được xây dựng với tầng Frontend dùng HTML, CSS, JavaScript thuần; tầng Backend dùng Node.js cùng Express framework; tầng Database sử dụng hệ quản trị MongoDB (Mongoose ORM). Cuối cùng là bước kiểm thử các RESTful API và triển khai vận hành thực tế trên nền tảng Cloud (Vercel, Render và MongoDB Atlas).

5. Tóm tắt nội dung báo cáo

Chương 1 trình bày bài toán, yêu cầu và cơ sở lý thuyết. Chương 2 mô tả phân tích, thiết kế, các sơ đồ và dữ liệu. Chương 3 trình bày việc xây dựng, API, kiểm thử và đánh giá. Chương 4 trình bày kiến trúc triển khai Cloud, cấu hình môi trường và minh chứng vận hành thực tế.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Bối cảnh và mô tả bài toán

Trong hoạt động giáo dục và đào tạo hiện nay, việc tổ chức lớp học, phân phối bài giảng, theo dõi tiến độ học tập và ghi nhận thông tin đăng ký khóa học luôn liên quan đến nhiều khâu quản lý. Khi thông tin khóa học và dữ liệu học viên được lưu trữ rời rạc hoặc quản lý bằng các công cụ thủ công truyền thống, học viên sẽ khó chủ động trong việc tìm kiếm và theo dõi lộ trình học tập cá nhân. Về phía nhà quản lý, họ phải tốn nhiều thời gian xử lý thủ công các thao tác lặp lại như cấp quyền truy cập, đối soát số dư/thanh toán, xác nhận ghi danh, đồng thời gặp khó khăn trong việc nắm bắt tổng quan dữ liệu tương tác và doanh thu từ các khóa học.

Hệ thống Quản lý Khóa học Trực tuyến (Course Management System) được xây dựng như một sản phẩm web full-stack nhằm số hóa luồng nghiệp vụ cốt lõi của một nền tảng e-learning. Hệ thống hướng đến môi trường giáo dục trực tuyến giả lập, tập trung vào việc quản lý thống nhất danh mục khóa học, nội dung bài học, giao dịch nạp tiền, tiến độ ghi danh (enrollment) và quản lý tài khoản người dùng. Sản phẩm này không nhằm mục đích thay thế ngay lập tức các nền tảng giáo dục quy mô lớn, mà đóng vai trò là một mô hình học thuật thực tiễn nhằm chứng minh năng lực phân tích, thiết kế hệ thống web phân tầng (Client - Server) và khả năng triển khai vận hành ứng dụng trên nền tảng Cloud.

1.2. Mục tiêu hệ thống

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống quản lý khóa học trực tuyến vận hành được trên Internet, có kiến trúc tách lớp Frontend - Backend rõ ràng, hỗ trợ đăng nhập và phân quyền bảo mật, sử dụng cơ sở dữ liệu phi quan hệ (NoSQL) và đảm bảo luồng nghiệp vụ xuyên suốt giữa các vai trò. Sản phẩm phải mang lại trải nghiệm thao tác trực quan, đảm bảo người dùng ở mỗi vai trò chỉ truy cập và thao tác đúng với khu vực chức năng dành riêng cho quyền hạn của mình.

Mục tiêu	Nội dung thực hiện
Quản lý khóa học và ghi danh	Học viên có thể tìm kiếm khóa học theo nhu cầu; hệ thống cho phép xem chi tiết, quản lý số dư ví nội bộ (balance/transaction) và tiến hành ghi danh (enroll) vào khóa học.

Số hóa tiến trình học tập	Học viên truy cập và tham gia các bài học (lessons) thuộc khóa học đã đăng ký; hệ thống ghi nhận trạng thái và hỗ trợ tính năng đánh giá (review) nội dung học tập.
Bảo mật truy cập	Cung cấp chức năng đăng ký, đăng nhập, sử dụng JWT (JSON Web Token) để xác thực và kiểm soát quyền nghiêm ngặt theo vai trò (User/Admin) tại backend.
Quản trị dữ liệu	Admin có toàn quyền quản lý tài khoản người dùng, tạo lập/chỉnh sửa/xóa các khóa học, xây dựng nội dung bài học và theo dõi tổng quan các giao dịch trên hệ thống.
Triển khai Cloud	Frontend được cấu hình triển khai qua Vercel, backend triển khai qua các dịch vụ Cloud (như Render) và sử dụng MongoDB Atlas cho database nhằm cung cấp URL vận hành thực tế.

Bảng 1.2.Mục tiêu hệ thống

1.3. Đối tượng sử dụng

Nhóm người dùng	Vai trò trong hệ thống	Giá trị nhận được
Học viên (User)	Đăng ký, đăng nhập, nạp số dư ví nội bộ, tìm kiếm khóa học, thực hiện ghi danh (enroll), tham gia các bài học và đánh giá (review) khóa học.	Chủ động lựa chọn khóa học, quản lý lộ trình học tập, theo dõi tiến độ một cách trực quan và dễ dàng tiếp cận kiến thức.
Quản trị viên (Admin)	Quản lý toàn bộ danh mục hệ thống bao gồm: người dùng, khóa học, bài học, kiểm soát các giao dịch (transactions) và xem tổng quan dữ liệu.	Tối ưu hóa quy trình quản trị nội dung giảng dạy, dễ dàng theo dõi doanh thu/giao dịch và kiểm soát hoạt động của học viên.

Bảng 2.Đối tượng sử dụng

1.4. Phạm vi thực hiện

Phạm vi của dự án bao gồm các chức năng cốt lõi: đăng ký/đăng nhập, quản lý hồ sơ và số dư cá nhân, hiển thị danh mục khóa học, quản lý bài học, thực hiện ghi danh và thanh toán học phí qua ví nội bộ, ghi nhận tiến độ học tập, đánh giá khóa học và chức năng quản trị danh mục tổng thể. Hệ thống phân chia thành 02 vai trò chính (User và Admin). Các phần không thuộc phạm vi thực hiện của đề tài bao gồm: tích hợp cổng thanh toán trực tuyến thực tế (như VNPay, MoMo), phòng học trực tuyến (livestreaming), hệ thống thi trắc nghiệm tính điểm tự động và ứng dụng trên thiết bị di động (mobile native).

1.5. Khảo sát yêu cầu

1.5.1. Yêu cầu chức năng

Mã	Nhóm chức năng	Mô tả ngắn
FR-01	Xác thực	Người dùng đăng ký tài khoản; hệ thống cho phép đăng nhập/đăng xuất và điều hướng giao diện theo vai trò (User/Admin).
FR-02	Quản lý hồ sơ	Học viên có thể xem thông tin cá nhân, cập nhật mật khẩu và theo dõi số dư ví hiện tại.
FR-03	Quản lý khóa học	Admin tạo mới, chỉnh sửa, xóa khóa học. Học viên xem danh sách, tìm kiếm và xem chi tiết khóa học.
FR-04	Quản lý bài học	Admin thêm, sửa, xóa bài học (lesson) thuộc một khóa học cụ thể. Học viên truy cập xem nội dung bài học.
FR-05	Ghi danh & Giao dịch	Học viên sử dụng số dư ví để ghi danh khóa học. Hệ thống tự động trừ tiền, tạo lịch sử giao dịch (Transaction) và xác nhận ghi danh (Enrollment).
FR-06	Đánh giá khóa học	Học viên có thể để lại nhận xét, đánh giá (Review) cho khóa học mà mình đã tham gia.
FR-07	Quản trị hệ thống	Admin có toàn quyền quản lý dữ liệu User, Course, Lesson, Transaction, Review và xem Dashboard tổng quan.

Bảng 1.5.1. Yêu cầu chức năng

1.5.2. Yêu cầu phi chức năng

Nhóm	Yêu cầu
Bảo mật	Không lưu mật khẩu dạng rõ; sử dụng bcrypt để mã hóa (hash). Token JWT được sử dụng để xác thực phiên làm việc. API backend có middleware kiểm tra tính hợp lệ của JWT và phân quyền User/Admin.
Hiệu năng	Tách biệt hoàn toàn tầng giao diện (Frontend) và tầng API (Backend). Có thể áp dụng phân trang (pagination) cho các danh sách dữ liệu dài.
Khả dụng	Ứng dụng được triển khai thành công trên môi trường Cloud, có thể truy cập vận hành thực tế thông qua các URL công khai với giao thức HTTPS.
Tính đúng đắn	Đảm bảo tính toàn vẹn khi ghi danh: chỉ người dùng có đủ số dư mới có thể ghi danh, ngăn chặn việc đăng ký trùng một khóa học nhiều lần.
Dễ sử dụng	Giao diện trực quan, chia rõ khu vực chức năng giữa Admin và User, có phản hồi thông báo (Alert/Toast) rõ ràng khi thao tác thành công hay thất bại.
Mở rộng	Cấu trúc code Node.js/Express và Schema của Mongoose được tổ chức theo chuẩn MVC, dễ dàng bổ sung tính năng thanh toán thực tế hoặc module thống kê về sau.

Bảng 1.5.2. Phi chức năng

1.6. Cơ sở lý thuyết

1.6.1. Kiến trúc 03 tầng

Hệ thống được thiết kế theo mô hình Client - Server tách biệt, phân chia ứng dụng thành tầng trình bày (Frontend - Client) và tầng nghiệp vụ/dữ liệu (Backend - Server). Tầng giao diện sử dụng HTML/CSS/JavaScript thuần nhận thao tác người dùng và hiển thị dữ liệu. Tầng nghiệp vụ (Node.js/Express) đóng gói logic xử lý, xác thực và cung cấp API. Cách tổ chức này làm rõ trách nhiệm, giảm sự phụ thuộc chặt chẽ giữa giao diện và cơ sở dữ liệu, đồng thời cho phép hai tầng phát triển và triển khai độc lập.

1.6.2. RESTful API

RESTful API dùng HTTP method để thể hiện ý nghĩa thao tác: GET để đọc, POST để tạo, PATCH để cập nhật trạng thái hoặc một phần dữ liệu, DELETE để xóa mềm/tắt dữ liệu theo thiết kế. Dữ liệu trao đổi giữa Next.js và NestJS ở định dạng JSON, sử dụng base path /api/v1 để thuận lợi cho versioning.

1.6.3. Authentication và Authorization

Authentication trả lời câu hỏi “người dùng là ai”, còn Authorization trả lời “người dùng được phép làm gì”. Hệ thống xác thực bằng email và mật khẩu; backend so sánh chuỗi hash thông qua bcrypt, tạo token JWT (JSON Web Token) và trả về cho client. Tại backend, các Middleware (như protect và authorize) sẽ đọc JWT từ header của request để kiểm tra tính hợp lệ và xác minh vai trò (role) trước khi cho phép người dùng truy cập vào các API nghiệp vụ.

1.6.4. Thiết kế cơ sở dữ liệu quan hệ

Đề tài sử dụng hệ quản trị cơ sở dữ liệu MongoDB (thông qua Mongoose ORM) để biểu diễn các thực thể nghiệp vụ: User, Course, Lesson, Enrollment, Transaction, và Review. Việc liên kết giữa các thực thể được thực hiện thông qua ObjectId (ví dụ: Enrollment tham chiếu tới User và Course; Lesson tham chiếu tới Course). Mô hình NoSQL mang lại sự linh hoạt cao trong việc thay đổi cấu trúc dữ liệu theo quá trình phát triển dự án.

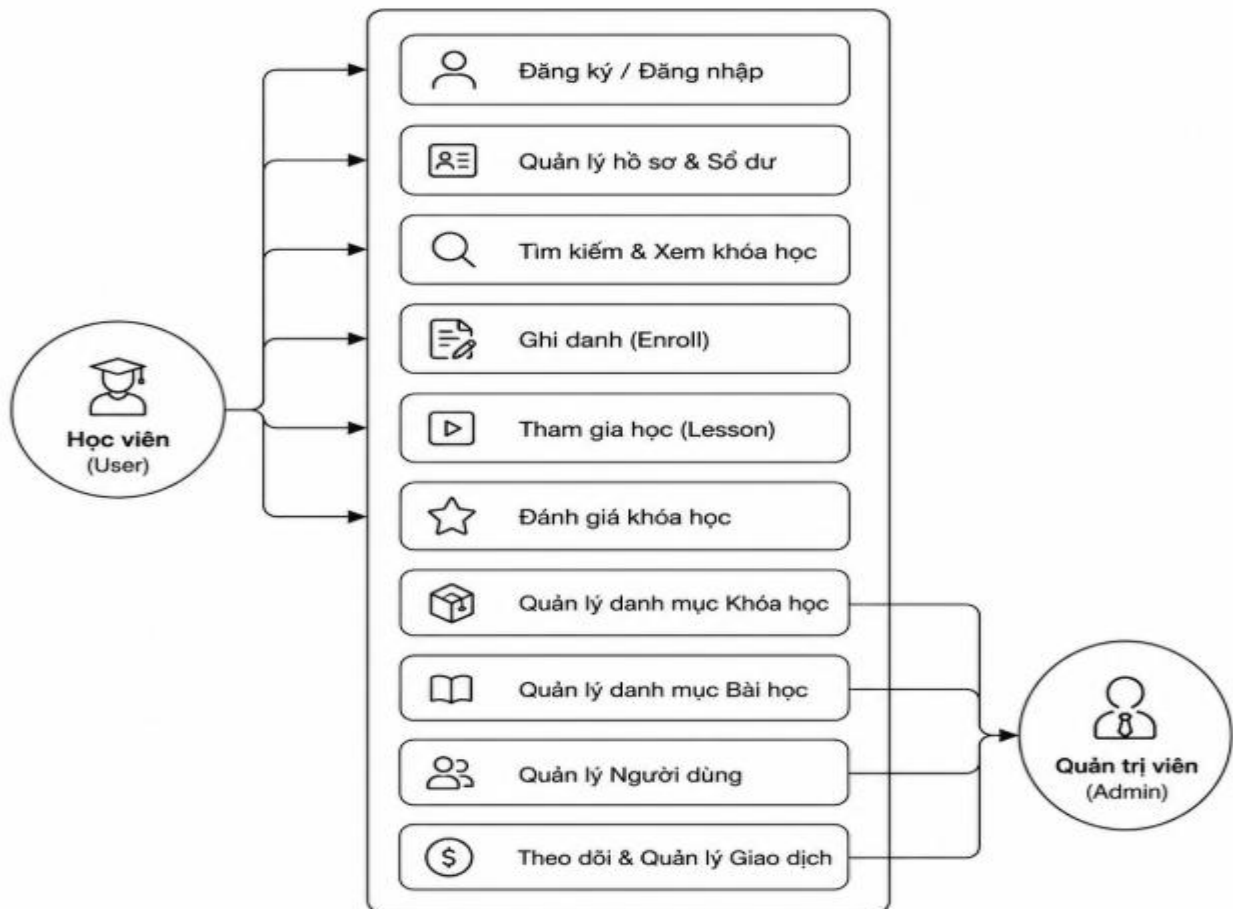
1.6.5. Cloud Computing

Cloud Computing cho phép triển khai Frontend, Backend và Database trên hạ tầng Internet thay vì chỉ chạy nội bộ trên localhost. Trong đề tài, Vercel đảm nhiệm việc hosting cho Frontend tĩnh, Backend Node.js có thể được chạy trên các nền tảng Web Service (như Render, Heroku...) và MongoDB Atlas cung cấp cụm cơ sở dữ liệu đám mây. Mô hình này đáp ứng hoàn toàn yêu cầu triển khai một ứng dụng web full-stack lên Internet của học phần.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích tác nhân và Use Case

Hệ thống có hai tác nhân chính tương ứng với hai vai trò (role) được định nghĩa trong dữ liệu User: Học viên (User) và Quản trị viên (Admin). Mỗi tác nhân được hệ thống cung cấp một tập hợp các chức năng chuyên biệt để thực hiện mục đích sử dụng của mình, đồng thời bị giới hạn quyền truy cập để đảm bảo tính bảo mật và toàn vẹn dữ liệu. Cụ thể, Học viên có thể xem thông tin, sử dụng số dư cá nhân để ghi danh và tham gia học tập; trong khi đó, Admin đóng vai trò là nhà cung cấp nội dung, được phép quản trị toàn bộ danh mục khóa học, bài học và tài khoản nhưng không tham gia vào luồng ghi danh học tập cá nhân.



Hình 2.1. Use Case tổng quát

2.2. Ma trận phân quyền

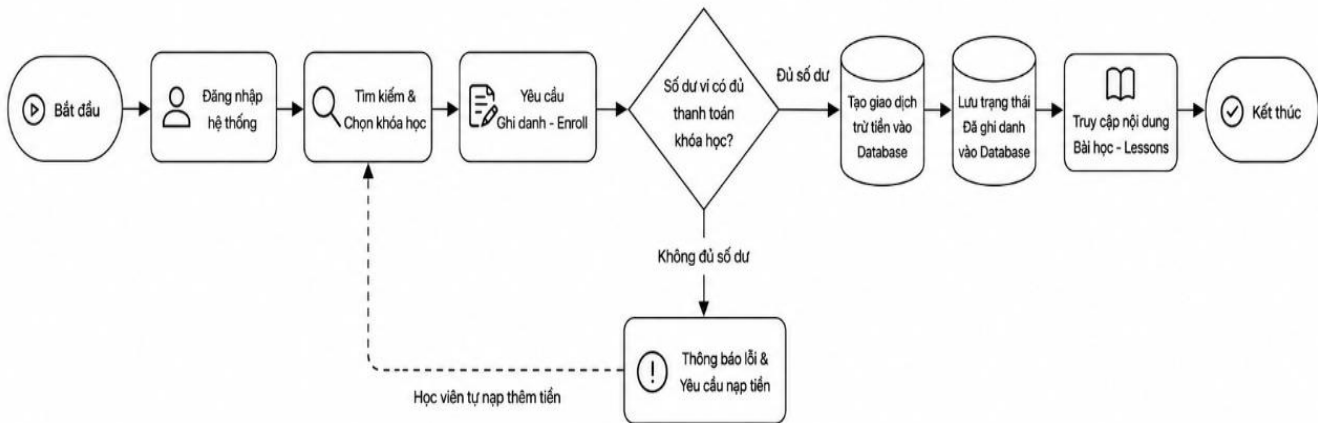
Tính năng	Học viên	ADMIN
Đăng ký/đăng nhập	Có (Đăng ký & Đăng nhập)	Chỉ Đăng nhập (Tài khoản được cấp)
Quản lý hồ sơ & Số dư	Có (Chỉ xem và cập nhật của bản thân)	Quản lý toàn bộ hồ sơ User
Xem danh mục khóa học	Xem danh sách và chi tiết các khóa công khai	Toàn quyền thêm, sửa, xóa khóa học
Ghi danh (Enrollment)	Có (Ghi danh bằng số dư ví cá nhân)	Lịch/hàng đợi của mình
Xem nội dung bài học	Chỉ xem các bài học thuộc khóa đã ghi danh	Toàn quyền thêm, sửa, xóa bài học
Giao dịch (Transaction)	Xem lịch sử giao dịch trừ tiền của mình	Theo dõi toàn bộ giao dịch trên hệ thống
Đánh giá (Review)	Có (Đánh giá các khóa học đã tham gia)	Có quyền xem và quản lý đánh giá

Bảng 2.2. Phân quyền

2.3. Quy trình nghiệp vụ chính

Luồng nghiệp vụ cốt lõi của hệ thống bắt đầu bằng việc Quản trị viên (Admin) thiết lập dữ liệu nền: tạo mới khóa học, thiết lập thông tin giá cả và thêm các bài học (lesson) tương ứng vào khóa học. Về phía người học, quy trình diễn ra theo các bước sau: Học viên đăng ký tài khoản và đăng nhập. Sau khi nạp tiền/có số dư trong hệ thống, học viên tìm kiếm và xem chi tiết khóa học. Khi quyết định học, học viên thực hiện chức năng Ghi danh (Enroll). Hệ thống sẽ tự động kiểm tra số dư, nếu hợp lệ sẽ tạo một Giao dịch (Transaction) để trừ tiền, đồng thời ghi nhận bản ghi Ghi danh (Enrollment) thành công. Kể từ thời điểm này, học viên được cấp quyền truy cập vào các tài nguyên bài học của khóa học đó.

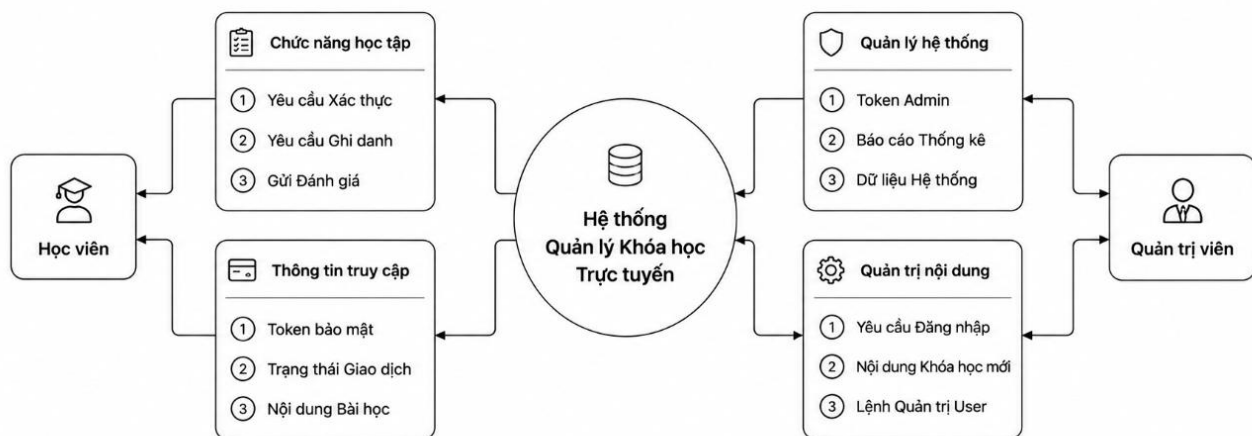
Khi hoàn tất quá trình học, học viên có thể thực hiện Đánh giá (Review) để phản hồi về chất lượng khóa học.



Hình 2.2. Luồng trạng thái ghi danh và tham gia khóa học của học viên

2.4. Sơ đồ luồng dữ liệu DFD mức ngữ cảnh

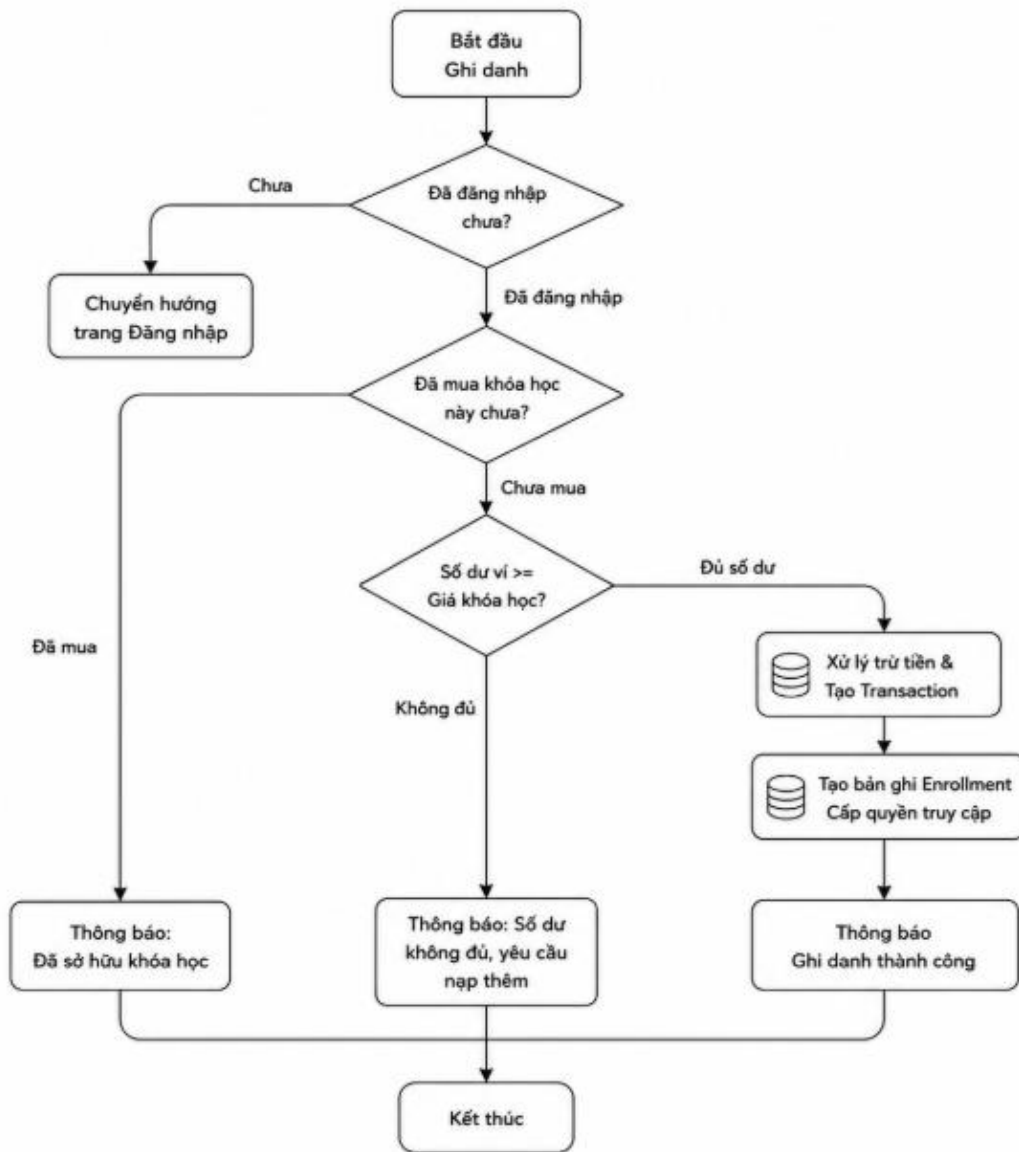
Sơ đồ DFD mức ngữ cảnh mô tả bức tranh tổng thể về cách các tác nhân bên ngoài (Học viên, Quản trị viên) tương tác với hệ thống. Mọi thao tác yêu cầu nội dung khóa học, gửi đánh giá hay xử lý giao dịch ghi danh đều được gửi đến hệ thống trung tâm. Hệ thống không cho phép người dùng truy cập trực tiếp vào cơ sở dữ liệu MongoDB; thay vào đó, mọi yêu cầu đều phải đi qua Backend (Node.js) để áp dụng các cơ chế xác thực JWT, phân quyền theo vai trò và kiểm tra các quy tắc nghiệp vụ (ví dụ: kiểm tra số dư trước khi ghi danh) trước khi lưu trữ hoặc trả về dữ liệu.



Hình 2.3. Sơ đồ luồng dữ liệu mức ngữ cảnh

2.5. Sơ đồ hoạt động quy trình ghi danh

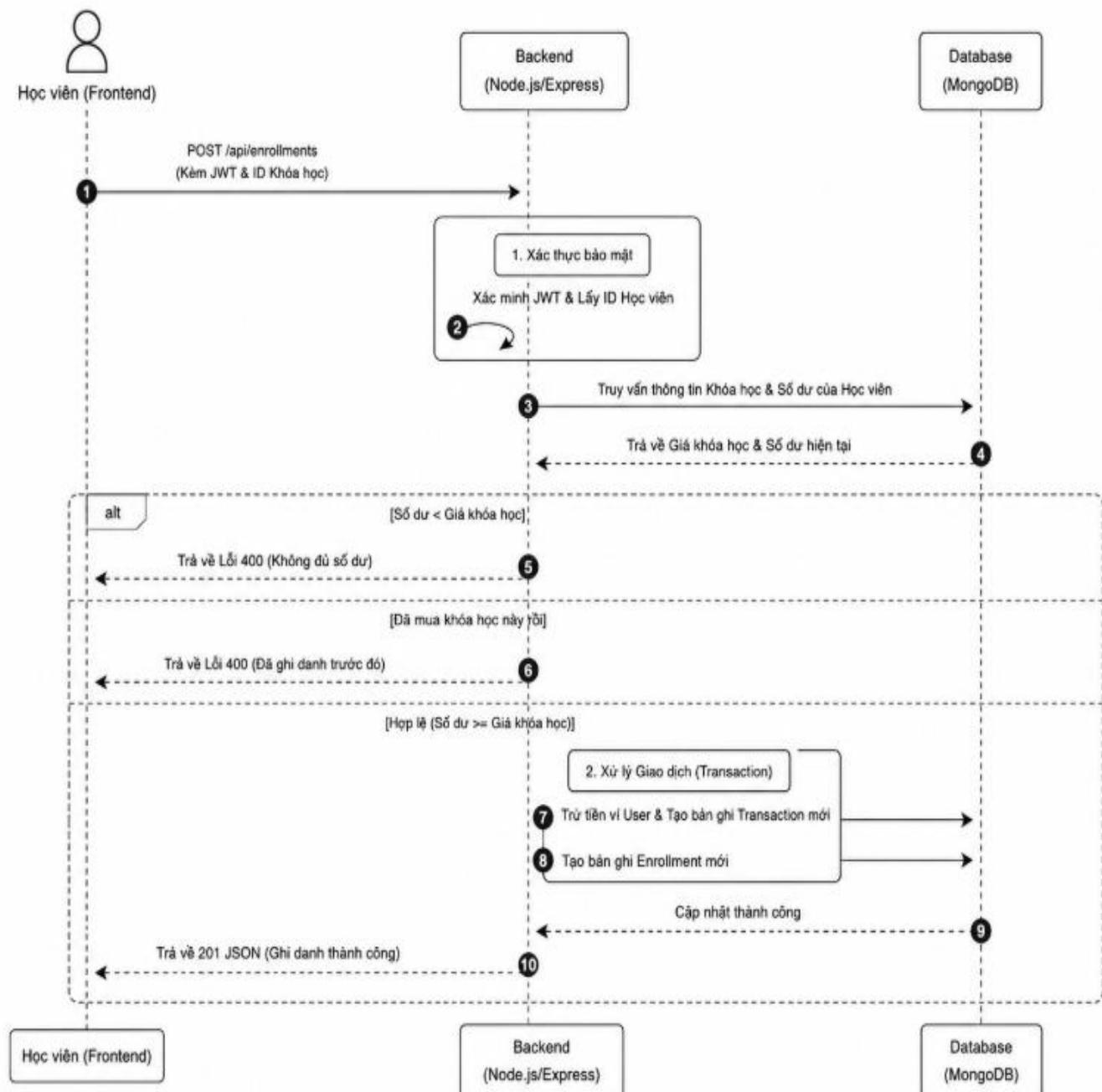
Sơ đồ hoạt động làm rõ nhánh kiểm tra điều kiện tài chính (số dư ví) trước khi xác nhận ghi danh. Khi số dư ví không đủ so với giá khóa học, hệ thống từ chối và yêu cầu người dùng nạp thêm tiền; khi số dư hợp lệ, backend tiến hành trừ tiền và tạo bản ghi Ghi danh (Enrollment) thành công, cấp quyền truy cập khóa học cho học viên.



Hình 2.4. Sơ đồ hoạt động quy trình ghi danh

2.6. Sơ đồ tuần tự quy trình ghi danh

Sơ đồ tuần tự mô tả chuỗi giao tiếp từ thao tác click chuột trên giao diện (Frontend) đến các API (Backend) và Cơ sở dữ liệu (MongoDB). Backend đóng vai trò trung tâm: kiểm tra tính hợp lệ của Token (JWT) để xác thực người dùng, kiểm tra số dư hiện tại và kiểm tra xem người dùng đã từng mua khóa học này chưa. Nếu thỏa mãn, hệ thống tạo bản ghi Transaction và Enrollment; sau đó trả về chuỗi JSON để frontend hiển thị thông báo ghi danh thành công.



Hình 2.5. Sơ đồ tuần tự quy trình ghi danh

2.7. Quy tắc nghiệp vụ cốt lõi

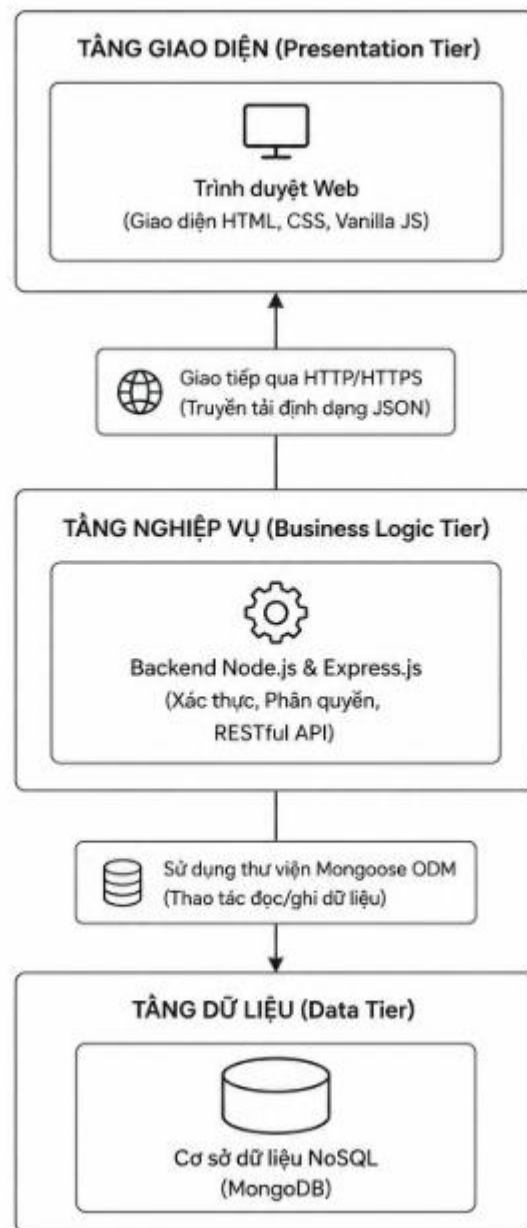
Mã	Quy tắc	Ý nghĩa
BR-01	Điều kiện ghi danh: Học viên chỉ được phép ghi danh khi số dư ví (Balance) lớn hơn hoặc bằng giá tiền của khóa học.	Đảm bảo hệ thống vận hành đúng luồng thanh toán, tránh việc ghi danh không gây thất thoát.
BR-02	Ràng buộc trùng lặp: Mỗi học viên chỉ được ghi danh một khóa học duy nhất một lần. Nếu khóa học đã tồn tại trong danh sách Enrollment của người dùng, chặn các yêu cầu ghi danh lại.	Tránh việc người dùng vô tình bấm mua nhiều lần và bị trừ tiền oan cho cùng một nội dung.
BR-03	Truy cập nội dung: Chỉ những tài khoản có trạng thái đã ghi danh (Enrolled) vào khóa học tương ứng mới được phép gọi API lấy chi tiết các Bài học (Lessons) bên trong.	Bảo vệ nội dung số (bản quyền khóa học), ngăn chặn truy cập trái phép từ những người chưa trả phí.
BR-04	Điều kiện đánh giá (Review): Học viên chỉ được tạo bài đánh giá và chấm điểm cho một khóa học sau khi họ đã ghi danh thành công khóa học đó.	Đảm bảo tính minh bạch và chân thực của các lượt đánh giá trên hệ thống.
BR-05	Tính toán vận giao dịch: Mỗi khi ghi danh thành công bắt buộc phải sinh ra một mã giao dịch (Transaction) để đối soát. Dữ liệu Transaction là bất biến (không thể sửa/xóa sau khi tạo).	Hỗ trợ việc thống kê, đối soát doanh thu một cách chính xác, minh bạch dành cho Quản trị viên.

Bảng 2.7. Quy tắc nghiệp vụ cốt lõi

2.8. Thiết kế kiến trúc tổng thể

Hệ thống được triển khai theo mô hình Client-Server phân tách rõ ràng. Tầng Frontend (chạy trên trình duyệt) không bao giờ kết nối trực tiếp với cơ sở dữ liệu; mọi thao tác truy vấn và cập nhật dữ liệu đều phải gọi qua các RESTful API của tầng Backend. Backend đóng vai trò như một chốt chặn bảo mật, chịu trách nhiệm xác minh danh tính (Authentication), phân quyền (Authorization), kiểm tra dữ liệu đầu vào và sử dụng thư viện Mongoose ODM để tương tác với MongoDB. Cách tách lớp này giúp bảo

mật chuỗi kết nối Database và tạo điều kiện thuận lợi để mở rộng các nền tảng khác (như Mobile App) về sau



Hình 2.6. Kiến trúc 03 tầng

2.9. Thiết kế frontend

Tầng giao diện (Frontend) được phát triển bằng HTML, CSS, và Vanilla JavaScript thuần, không phụ thuộc vào các framework phức tạp. Các tệp tin được phân chia rõ ràng theo từng tính năng nghiệp vụ: `index.html` (trang chủ), `course.html` (thông tin khóa học), `learn.html` (giao diện học tập), `profile.html` (hồ sơ và số dư) và các trang

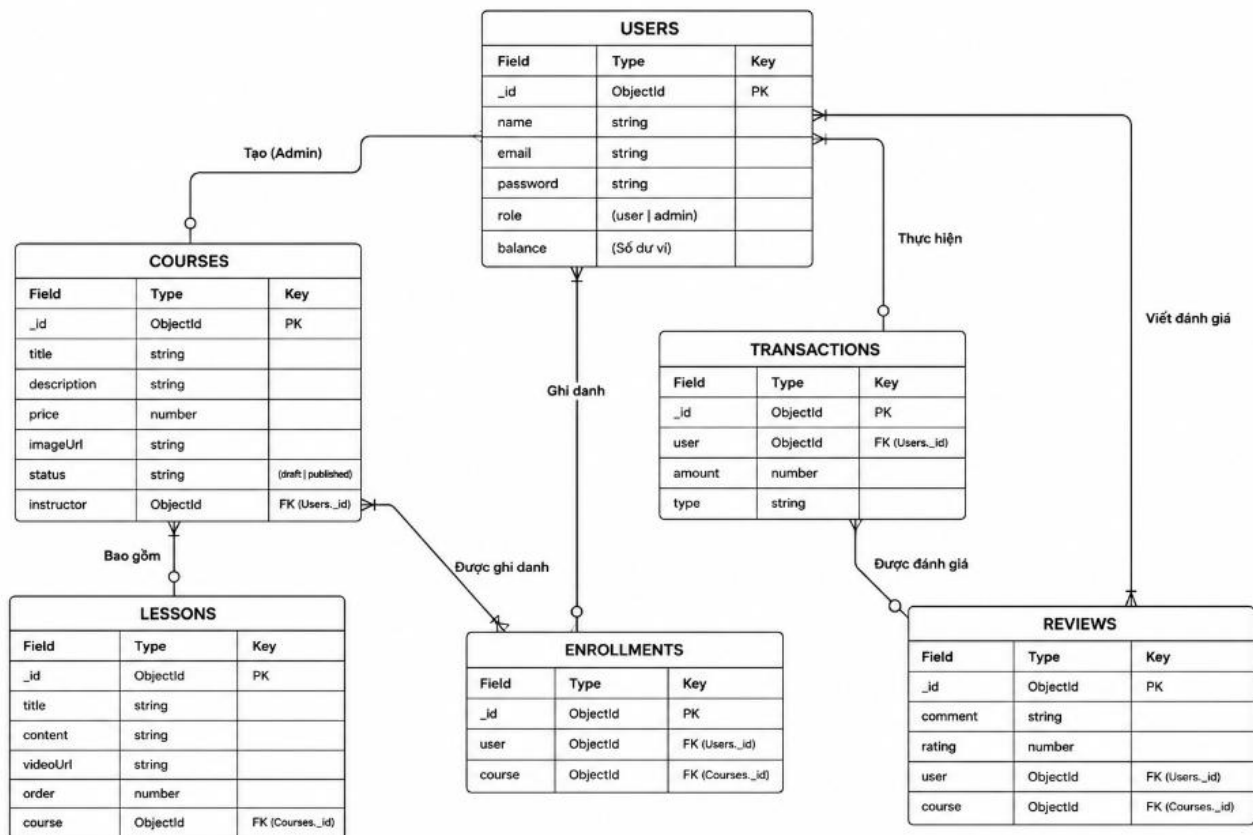
dành riêng cho quản trị viên (admin.html, admin-lessons.html). Các thành phần như Header, Sidebar, Card hiển thị khóa học được định dạng đồng nhất qua CSS dùng chung. JavaScript sử dụng hàm fetch() tiêu chuẩn của trình duyệt để gọi API Backend, đồng thời tự động đính kèm Token JWT vào header Authorization ở các endpoint yêu cầu quyền truy cập. Giao diện hỗ trợ responsive để hiển thị tốt trên cả màn hình máy tính và thiết bị di động.

2.10. Thiết kế backend

Tầng nghiệp vụ (Backend) sử dụng Node.js kết hợp cùng framework Express.js. Mã nguồn được tổ chức theo kiến trúc Controller-Route nhằm tách biệt các nhóm logic: Routes: Định nghĩa các endpoint phân chia theo tài nguyên, ví dụ: /api/users, /api/courses, /api/lessons, /api/enrollments. Controllers: Chứa logic nghiệp vụ (thêm, sửa, xóa, tìm kiếm). Middleware: Nổi bật là middleware xác thực Token (protect), đảm bảo chỉ những request hợp lệ mới được đi qua để gọi tới hàm xử lý trong Controller. Models: Khai báo cấu trúc Schema bằng Mongoose để chuẩn hóa dữ liệu trước khi lưu vào MongoDB.

2.11. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu phi quan hệ (NoSQL) MongoDB. Dữ liệu được tổ chức dưới dạng các document lưu trong các Collection. Collection users là cốt lõi quản lý thông tin tài khoản và số dư ví. Collection courses lưu trữ danh mục khóa học; bên trong mỗi khóa học sẽ tham chiếu tới nhiều lessons. Hoạt động tương tác của người học với nền tảng được ghi nhận thông qua các collection: enrollments (lịch sử ghi danh), transactions (biến động số dư ví) và reviews (đánh giá). Dù là NoSQL, các collection vẫn được liên kết chặt chẽ thông qua trường khóa ngoại ObjectId (ví dụ: Lesson.course trỏ đến Course._id) để đảm bảo tính toàn vẹn dữ liệu.



Hình 2.7. ERD rút gọn

2.12. Data Dictionary rút gọn

Bảng	Mục đích	Các thuộc tính quan trọng	Liên kết (Ref / FK)
users	Quản lý tài khoản, phân quyền và số dư ví nội bộ.	<u>_id</u> , name, email, password (hash), role, balance (mặc định 0).	1-n với enrollments, transactions, reviews.
courses	Quản lý thông tin chung và giá trị của khóa học.	<u>_id</u> , title, description, price, imageUrl, status.	Tham chiếu (ref) tới instructor (User). 1-n với lessons.
lessons	Lưu trữ nội dung chi tiết bài học thuộc khóa học.	<u>_id</u> , title, content, videoUrl, order (thứ tự bài giảng).	Tham chiếu (ref) tới course (Course).

enrollments	Lưu lịch sử mở khóa quyền truy cập khóa học.	_id, thời gian ghi danh (timestamps).	Tham chiếu tới user và course.
transactions	Lưu vết lịch sử nạp/trừ tiền trong ví người dùng.	_id, amount, kiểu giao dịch (type), thời gian.	Tham chiếu tới user.
reviews	Lưu trữ phản hồi chất lượng khóa học từ người học.	_id, rating (điểm số), comment (nội dung).	Tham chiếu tới user và course.

Bảng 2.12 Data Dictionary

2.13. Thiết kế giao diện và luồng thao tác

Thiết kế giao diện người dùng (UI) ưu tiên luồng thao tác thực dụng theo nhiệm vụ:

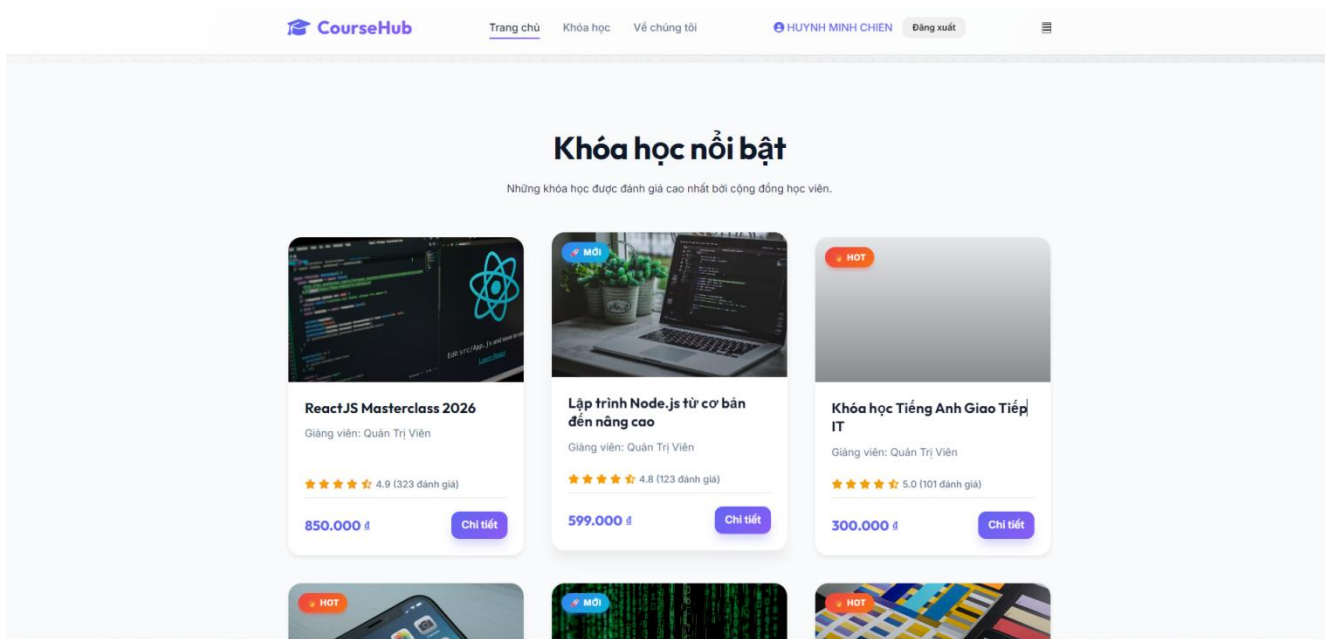
Đối với Học viên (User): Khi truy cập hệ thống, học viên được hướng sự chú ý ngay vào trang hiển thị Danh sách Khóa học. Quá trình bấm xem chi tiết, yêu cầu Ghi danh bằng số dư và chuyển tiếp vào màn hình Học bài (Learn) diễn ra trơn tru. Hồ sơ cá nhân (Profile) được đặt riêng để tập trung quản lý số dư (Balance) và thông tin.

Đối với Quản trị viên (Admin): Sau khi đăng nhập, hệ thống điều hướng thẳng vào Dashboard (Bảng điều khiển) ẩn. Tại đây, Admin thao tác dưới dạng Data Table (bảng dữ liệu) để dễ dàng thêm, sửa, xóa khóa học và cập nhật nội dung bài học một cách nhanh chóng. Các hành động mang tính rủi ro làm thay đổi dữ liệu như Xóa khóa học hay Đăng xuất luôn được yêu cầu xác nhận (Confirm Dialog) để giảm thiểu rủi ro thao tác nhầm lẫn.

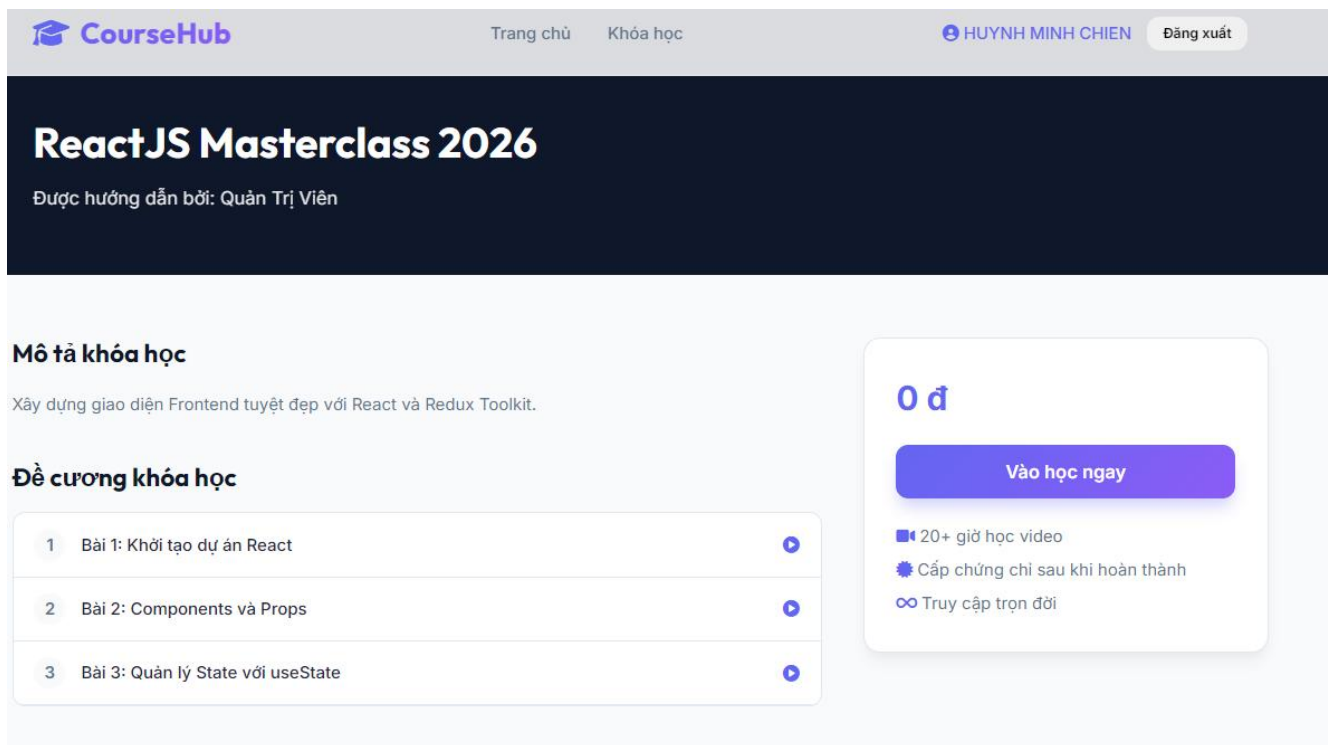
CHƯƠNG 3. XÂY DỰNG VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Xây dựng Frontend

Phần Frontend được xây dựng bằng HTML5, CSS3 và Vanilla JavaScript thuần, không phụ thuộc vào framework nhằm tối ưu hóa hiệu năng và dung lượng mã nguồn. Giao diện được chia thành các tệp HTML độc lập (index.html, login.html, course.html, learn.html, admin.html...), mỗi tệp tương ứng với một màn hình chức năng. Các thành phần tái sử dụng (như Header, Footer, Card Khóa học) được định dạng chung qua các file CSS. Đối với trải nghiệm người dùng, các Form đăng ký/đăng nhập đều được trang bị tính năng kiểm tra dữ liệu đầu vào (validation) hiển thị lỗi trực quan. Trong luồng tìm kiếm và ghi danh, học viên có thể lọc khóa học theo tên hoặc danh mục, sau đó bấm xem chi tiết trước khi quyết định ghi danh. Thiết kế này giúp người dùng dễ dàng tiếp cận nội dung bài giảng một cách nhanh chóng nhất.



Hình 3.1. Giao diện tổng quan



Hình 3.2. Giao diện chi tiết khóa học

3.2. Xây dựng Backend

Môi trường Node.js kết hợp framework Express.js được lựa chọn để xây dựng Backend nhờ đặc tính gọn nhẹ, xử lý bất đồng bộ tốt và hệ sinh thái phong phú. API được tổ chức theo chuẩn RESTful. Cấu trúc mã nguồn chia theo mô hình MVC: thư mục routes định vị các endpoint, thư mục controllers xử lý logic nghiệp vụ và tương tác với Database. Backend sử dụng package cors để cấp phép giao tiếp chéo miền (Cross-Origin) từ Frontend (Vercel), và package dotenv để quản lý các biến môi trường nhạy cảm (như JWT_SECRET, MONGO_URI). Ứng dụng đọc port động từ biến môi trường PORT để tương thích hoàn toàn với nền tảng triển khai Web Service như Render hoặc Heroku.

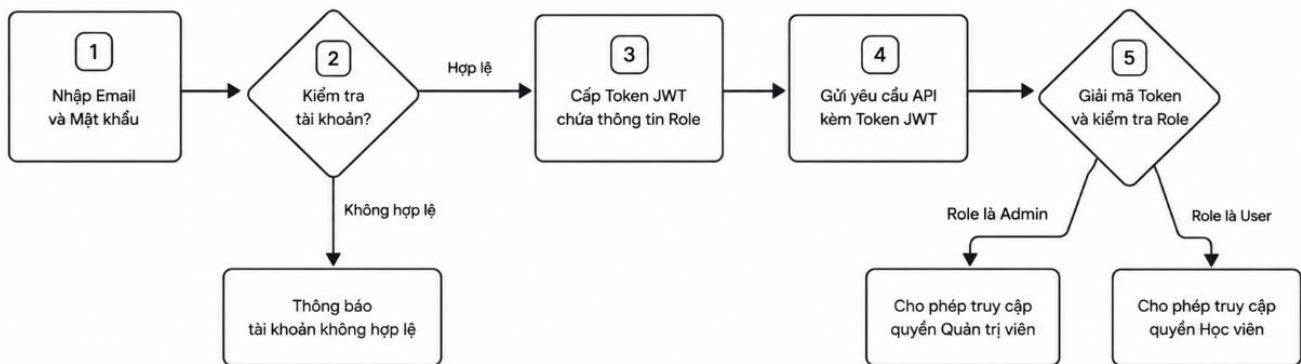
3.3. Xây dựng Database

Hệ quản trị cơ sở dữ liệu MongoDB (NoSQL) được sử dụng thông qua thư viện Mongoose ODM. Mongoose giúp định nghĩa các Schema rõ ràng cho User, Course, Lesson, Enrollment, Transaction kèm theo các quy tắc validation (như required, unique). Hệ thống đi kèm các kịch bản khởi tạo dữ liệu mẫu (các file seedCourses.js, seedLessons.js, setup_roles.js...). Khi chạy các script này, CSDL tự động được điền sẵn

thông tin tài khoản Admin, các khóa học demo và danh sách bài học chi tiết. Điều này chứng minh ứng dụng có thể kết nối thành công tới cụm cơ sở dữ liệu đám mây (MongoDB Atlas) và sẵn sàng vận hành.

3.4. Authentication và Authorization

Khi người dùng gửi email và mật khẩu, Backend tìm tài khoản trong CSDL, sử dụng thư viện bcrypt để so sánh chuỗi hash mật khẩu. Nếu hợp lệ, hệ thống tạo ra một Token (JWT) và trả về cho Frontend. Tại trình duyệt, JWT được lưu trữ cục bộ (LocalStorage/SessionStorage). Ở các thao tác yêu cầu bảo mật, Frontend sẽ đính kèm JWT này vào header Authorization (chuẩn Bearer Token). Tại Backend, các Middleware như protect sẽ giải mã Token để nhận diện người dùng; tiếp đó Middleware authorize('admin') sẽ kiểm tra Role (vai trò) để chặn đứng các tài khoản Học viên (User) cố tình truy cập vào các API quản trị (Admin).



Hình 3.3. Luồng đăng nhập, phân quyền RBAC

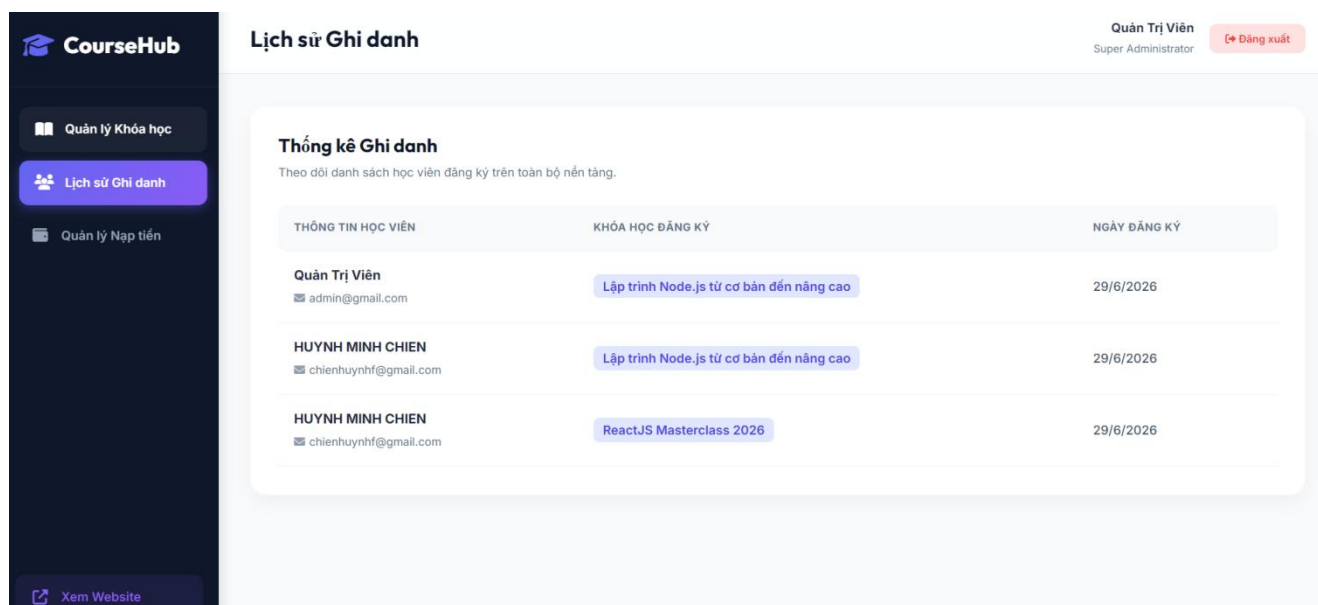
3.5. Các API endpoint tiêu biểu

Phương thức	Endpoint	Vai trò	Chức năng
POST	/api/auth/register	Public	Đăng ký tài khoản Học viên mới.
POST	/api/auth/login	Public	Đăng nhập và nhận Token JWT.
GET	/api/auth/me	Authenticated	Lấy thông tin user hiện tại và số dư ví.
GET	/api/courses	Public	Hiển thị danh sách khóa học (dành cho trang chủ).
POST	/api/enrollments/:courseId	User	Xử lý ghi danh, trừ số dư và cấp quyền truy cập.
GET	/api/lessons/course/:courseId	Enrolled User	Lấy danh sách bài học (chỉ dành cho tài khoản đã ghi danh).
POST/PUT/DELETE	/api/admin/courses	Admin	Quản trị viên Thêm/Sửa/Xóa dữ liệu khóa học.
GET	/api/health	Public	Kiểm tra trạng thái hoạt động của Server và Database.

Bảng 3.5 API tiêu biểu

3.6. Minh họa giao diện quản trị

Giao diện dành cho Quản trị viên (Admin) được thiết kế tách biệt (các trang admin.html, admin-lessons.html). Tại đây, Admin có thể theo dõi danh sách người dùng, thống kê các khóa học hiện có và thực hiện thêm mới nội dung. Các thao tác thêm Bài học (Lesson) được liên kết chặt chẽ với Khóa học (Course) tương ứng bằng các form điền liệu trực quan.



Hình 3.4. Giao diện tổng quan quản trị

3.7. Kiểm thử hệ thống

Kiểm thử được thực hiện nhằm đánh giá luồng nghiệp vụ từ Frontend đến Backend và Database. Các request API được gọi trực tiếp lên domain của Render. Dưới đây là các kịch bản minh chứng cho môi trường production:

STT	Kịch bản	Dữ liệu / Đầu vào	Kết quả mong đợi	Kết quả
TC-01	Khởi tạo dữ liệu (Seed)	Gọi script seed database	Courses và Lessons mẫu hiển thị trên DB Atlas.	Đạt
TC-02	Login Học viên	Nhập Email & Mật khẩu hợp lệ	Backend trả về JWT chứa role user.	Đạt

TC-03	Phân quyền RBAC	Dùng JWT user gọi API tạo Khóa học	Backend trả về lỗi 403 Forbidden (Không đủ quyền).	Đạt
TC-04	Ghi danh thiếu số dư	Bấm Ghi danh khi ví = 0	Trả về thông báo lỗi "Không đủ số dư".	Đạt
TC-05	Ghi danh hợp lệ	Bấm Ghi danh khi ví > Giá KH	Tạo Enrollment, trừ tiền ví, tạo Transaction.	Sẵn sàng test UI
TC-06	Truy cập bài học	Bấm xem video khóa học đã mua	Lấy thành công dữ liệu Lesson và Video URL.	Sẵn sàng test UI
STT	Kịch bản	Dữ liệu / Đầu vào	Kết quả mong đợi	Kết quả

Bảng 3.7. Kiểm thử

3.8. Đánh giá kết quả

3.8.1. Ưu điểm

- Triển khai thành công kiến trúc Client-Server tách biệt; Frontend và Backend hoạt động độc lập, giao tiếp chuẩn RESTful API.
- Luồng nghiệp vụ cốt lõi trơn tru: Từ xem danh mục, quản lý ví nội bộ, ghi danh đến truy cập bài học đều liên hoàn, sát với nghiệp vụ e-learning thực tế.
- Có cơ chế bảo mật (JWT, bcrypt) và phân quyền RBAC (User/Admin) đáp ứng tốt yêu cầu phân quyền hệ thống.
- Cấu trúc dữ liệu NoSQL (MongoDB) được tổ chức khoa học qua Mongoose, có ràng buộc ObjectId tham chiếu giữa các collection.
- Đã đưa sản phẩm lên Cloud thành công (Vercel cho Frontend tĩnh, Render cho Node.js API và MongoDB Atlas cho dữ liệu), cung cấp URL vận hành thực tế.

3.8.2. Hạn chế

- Ứng dụng hiện chỉ mô phỏng thanh toán qua "ví nội bộ" (Balance), chưa kết nối với các cổng thanh toán trực tuyến thực tế (VNPay, Momo, PayPal).
- Hệ thống trình phát video (Video Player) mới hỗ trợ các link tĩnh cơ bản (như YouTube iframe, MP4), chưa tích hợp cơ chế chống tải lậu video (DRM) hay Streaming HLS chuyên nghiệp.
- Gói Cloud miễn phí (Free Tier) trên Render có thể gặp tình trạng "ngủ đông" (Cold start) khiến lần gọi API đầu tiên mất nhiều thời gian chờ đợi.
- Chưa hỗ trợ chức năng làm bài tập trắc nghiệm (Quiz) chấm điểm tự động.

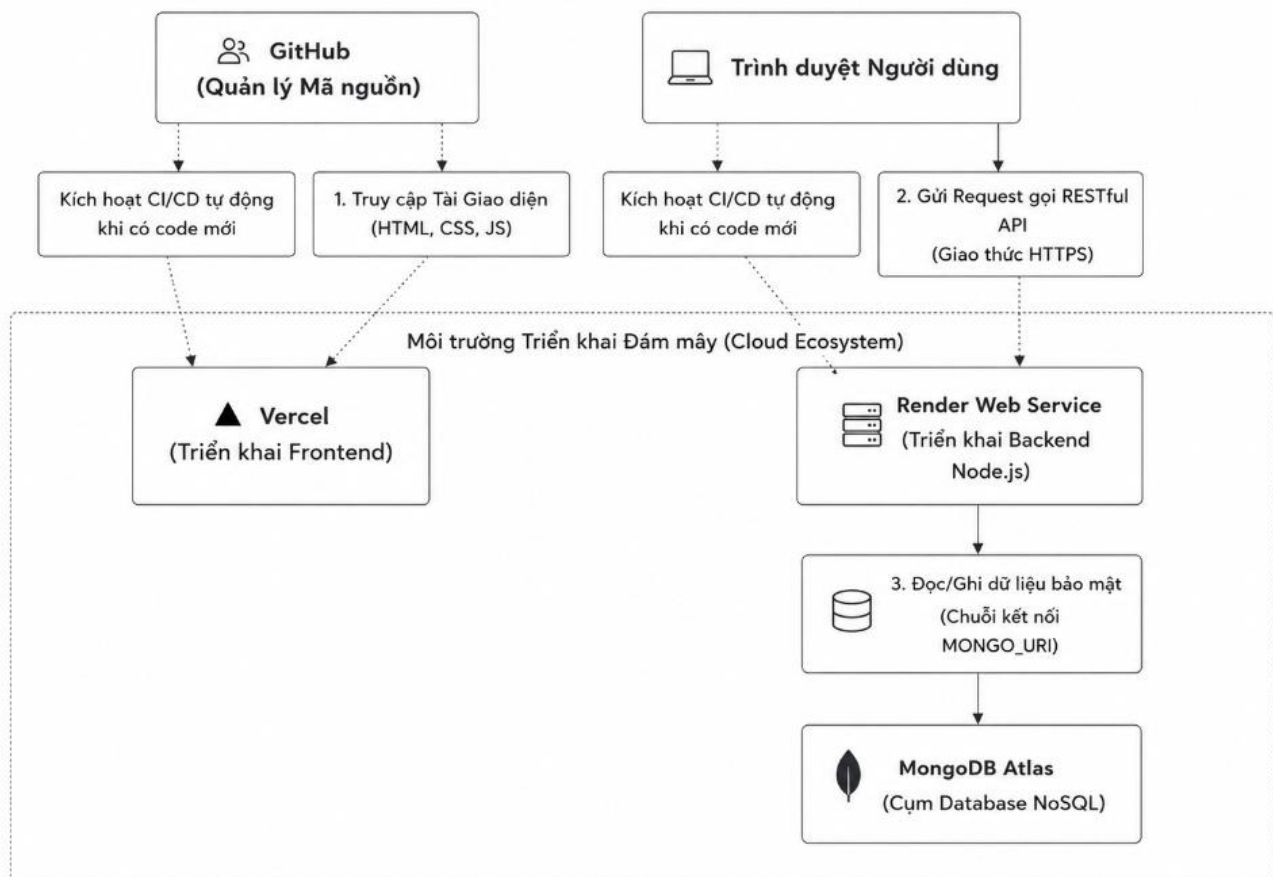
3.8.3. Hướng phát triển

- Tích hợp cổng thanh toán trực tuyến (VNPay/Stripe) để học viên có thể nạp tiền thật vào hệ thống thay vì thao tác nội bộ.
- Xây dựng module kiểm tra trắc nghiệm (Quiz) cuối mỗi khóa học và tự động cấp Chứng nhận (Certificate) dạng PDF khi học viên hoàn thành.
- Mở rộng tính năng tương tác: Chatbox hỗ trợ trực tuyến, phần Hỏi đáp (Q&A) bên dưới mỗi bài giảng giữa Học viên và Giảng viên.
- Tối ưu hóa UI/UX dành riêng cho nền tảng Mobile, tiến tới xây dựng ứng dụng Mobile App (React Native/Flutter) tái sử dụng lại các API Backend đã có.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG TRÊN CLOUD

4.1. Kiến trúc triển khai

Hệ thống Quản lý Khóa học Trực tuyến được triển khai với ba dịch vụ Cloud độc lập nhằm đảm bảo tính phân tán và dễ dàng mở rộng. Nền tảng Vercel chịu trách nhiệm phân phối ứng dụng Frontend (HTML/JS tĩnh) tới người dùng cuối. Nền tảng Render được sử dụng để chạy Backend Node.js dưới dạng Web Service và công khai các RESTful API. Dữ liệu hệ thống được lưu trữ và vận hành trên MongoDB Atlas (dịch vụ cơ sở dữ liệu NoSQL đám mây). Các thành phần giao tiếp với nhau thông qua giao thức bảo mật HTTPS/TLS, kết hợp biến môi trường để tách thông tin nhạy cảm (như mật khẩu DB, khóa bảo mật JWT) khỏi source code.



Hình 4.1. Kiến trúc triển khai Cloud thực tế

4.2. Cấu hình môi trường triển khai

hành phần	Nền tảng	Cấu hình chính	Vai trò
Frontend	Vercel	Root Directory: frontend; tự động xử lý file tĩnh (HTML/CSS/JS). Biến CLOUD_API_URL trong file config.js trỏ tới API của Render.	Build và phân phối giao diện web cho người học.
Backend	Render	Root Directory: backend; Môi trường: Node.js. Build command: npm install Start command: npm start	Chạy Express.js REST API, xử lý logic, JWT và kết nối MongoDB.
Database	MongoDB Atlas	Cụm (Cluster) cơ sở dữ liệu NoSQL đám mây; sử dụng chuỗi kết nối MONGO_URI.	Lưu trữ dữ liệu danh mục khóa học, người dùng và giao dịch.
Source code	GitHub	Repository chung (monorepo) quản lý code của cả frontend và backend.	Quản lý mã nguồn và kích hoạt CI/CD tự động lên Vercel/Render.

Bảng 4.2 cấu hình môi trường triển khai

4.3. Các bước triển khai

Bước 1: Tạo dự án (Cluster) trên MongoDB Atlas và lấy chuỗi kết nối. Giá trị này được lưu ở Environment Variables (Biến môi trường) của Render dưới tên MONGO_URI, tuyệt đối không commit lên GitHub.

Bước 2: Tạo Render Web Service kết nối từ repository GitHub. Chọn Root Directory là backend, chạy lệnh npm install để cài đặt thư viện (express, mongoose, jsonwebtoken...). Start command được thiết lập là npm start (tương đương node server.js).

Bước 3: Chạy kịch bản khởi tạo (Seed) một lần duy nhất bằng các lệnh node seedCourses.js, node seedLessons.js và node setup_roles.js để đầy danh mục khóa học, bài học và tài khoản admin mẫu lên cơ sở dữ liệu MongoDB Atlas.

Bước 4: Tạo Vercel project kết nối với cùng repository, chọn Root Directory là frontend. Nhờ logic tự động bắt môi trường trong config.js, Vercel sẽ gọi API đến https://chuyende1.onrender.com/api thay vì localhost.

4.4. Minh chứng cấu hình và vận hành

Build

Source
The build source for your Web Service

https://github.com/22050025-coder/do-an-web

Edit

Branch
The Git branch to build and deploy.

main

Edit

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

backend

Edit

Build Command
Render runs this command to build your app before each deploy.

backend/ \$ npm install

Edit

Git Credentials
User providing the credentials to pull the repository.

22050025@student.bdu.edu.vn (you)

Use My Credentials

Deploy

Pre-Deploy Command Optional
Render runs this command before the start command. Useful for database migrations and static asset uploads.

backend/ \$

Edit

Start Command
Render runs this command to start your app with each deploy.

backend/ \$ node server.js

Edit

Auto-Deploy
By default, Render automatically deploys your service whenever you update its code or configuration. Disable to handle deploys manually. [Learn more](#).

On Commit

Edit

Deploy Hook
Your private URL to trigger a deploy for this server. Remember to keep this a secret.

.....

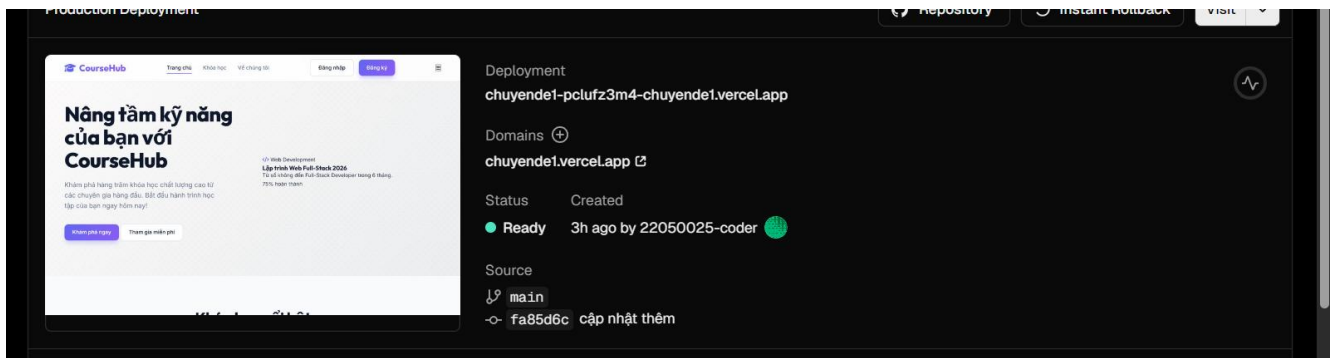
Regenerate hook

Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

Export Edit

KEY	VALUE	
JWT_SECRET		Ⓢ
MONGO_URI		Ⓢ

Hình 4.2. Cấu hình Build Command, Start Command và Free instance trên Render



Hình 4.3. Cấu hình Frontend trên Vercel

4.5. URL truy cập và kiểm thử

Hạng mục	Địa chỉ
Frontend production	https://chuyende1.vercel.app
Backend production	https://chuyende1.onrender.com
Repository Github	https://github.com/22050025-coder/do-an-web.git

Bảng 4.5 URL truy cập

Sau khi triển khai thành công, việc kiểm thử cho thấy Frontend đã gọi thành công danh sách Khóa học thông qua API GET /api/courses từ Render. Dữ liệu trả về (Count > 0) chứng minh backend Express đã kết nối ổn định tới MongoDB Atlas. Khi thử nghiệm đăng nhập bằng tài khoản Học viên, hệ thống trả về HTTP 200 OK cùng Token JWT; Token này được Frontend tiếp nhận và lưu an toàn vào LocalStorage, cho phép các thao tác phân quyền hoạt động mượt mà.

4.6. Bảo mật cấu hình triển khai

Toàn bộ các giá trị nhạy cảm như MONGO_URI (kết nối DB) và JWT_SECRET (khóa mã hóa Token) được thiết lập ẩn trong hệ thống quản lý Environment Variables của Render, không xuất hiện trong mã nguồn công khai (đã bỏ qua .env bằng .gitignore). Đối với Frontend, việc sử dụng kiến trúc tách biệt (Client-Server) giúp che giấu hoàn toàn các thông tin vận hành CSDL đối với trình duyệt. Mã thông báo xác thực JWT chỉ được cấp qua các phản hồi an toàn và được quy định gửi trong header Authorization

nhằm ngăn chặn các lỗ hổng giả mạo yêu cầu (CSRF) cơ bản. Giao thức HTTPS trên Vercel và Render đảm bảo dữ liệu trong quá trình truyền tải được mã hóa toàn vẹn.

4.7. Hạn chế của môi trường miễn phí

Môi trường triển khai miễn phí (Free Tier) phù hợp cho mục đích học thuật và demo vì không phát sinh chi phí hạ tầng. Tuy nhiên, dịch vụ Web Service của Render có cơ chế "ngủ đông" (Cold Start/Spin down) nếu hệ thống không nhận được request nào trong vòng 15 phút. Điều này dẫn đến tình trạng lần truy cập (hoặc đăng nhập) đầu tiên sau khoảng thời gian dài có thể mất tới 30-50 giây để server khởi động lại. Khi báo cáo hoặc demo trực tiếp với hội đồng, cần chủ động truy cập frontend và thực hiện một vài lời gọi API trước để dịch vụ trở lại trạng thái sẵn sàng. Nếu muốn vận hành nền tảng giáo dục thực tế phục vụ hàng ngàn học viên, cần phải mua các gói dịch vụ cao cấp (Paid Tier) để đảm bảo thời gian uptime 99.9%.

4.8. Source code

Mã nguồn của Hệ thống quản lý khóa học trực tuyến được quản lý trên GitHub tại địa chỉ: <https://github.com/22050025-coder/do-an-web.git>

KẾT LUẬN

Hệ thống Quản lý Khóa học Trực tuyến đã hoàn thành xuất sắc mục tiêu xây dựng một nền tảng web full-stack theo kiến trúc Client-Server, phục vụ trọn vẹn quy trình phân phối nội dung giáo dục và quản lý học tập. Hệ thống sử dụng giao diện Frontend bằng HTML/JS thuần, Backend bằng Node.js (Express) và cơ sở dữ liệu phi quan hệ MongoDB (Mongoose); hỗ trợ đầy đủ các tính năng: đăng ký, đăng nhập, phân quyền bảo mật (02 role: User và Admin), quản lý hồ sơ cá nhân, nạp/trừ số dư ví, quản lý danh mục khóa học, bài học, ghi danh (enrollment), theo dõi giao dịch và đánh giá chất lượng.

Điểm quan trọng và nổi bật của sản phẩm là sự liên kết logic giữa các vai trò trong một quy trình nghiệp vụ thống nhất, thay vì chỉ là các chức năng Thêm-Sửa-Xóa (CRUD) dữ liệu độc lập. Diễn hình là luồng nghiệp vụ Học viên đăng ký tài khoản -> Nạp số dư -> Tìm kiếm khóa học -> Hệ thống kiểm tra giao dịch trừ tiền -> Ghi nhận trạng thái Ghi danh -> Cấp quyền truy cập bài học và Đánh giá.

Bên cạnh đó, hệ thống đã được cấu hình và triển khai thành công lên môi trường Cloud thực tế bằng việc kết hợp 3 nền tảng: Vercel (Frontend), Render (Backend API) và MongoDB Atlas (Database). Các bài kiểm thử API, kiểm thử giao diện và xác thực phiên làm việc bằng JWT đều cho kết quả tốt. Sản phẩm đáp ứng toàn bộ các yêu cầu khắt khe của một đề tài tiểu luận lập trình Web: kiến trúc đa tầng rõ ràng, nhiều hơn 5 chức năng nghiệp vụ, có xác thực/phân quyền RBAC, xây dựng chuẩn RESTful API, thiết kế CSDL đa liên kết và có URL vận hành thực tế trên Internet.

Trong tương lai, đề tài hoàn toàn có thể phát triển sâu hơn thông qua việc: tích hợp cổng thanh toán trực tuyến thực (VNPay/Momo), xây dựng hệ thống làm bài thi trắc nghiệm tự động chấm điểm cấp chứng chỉ, bổ sung tính năng tương tác (Hỏi đáp, Chat) và gửi thông báo nhắc nhở học tập qua Email/SMS. Tuy vậy, với giới hạn hiện tại, phiên bản MVP này là một nền tảng cực kỳ phù hợp cho mục tiêu của học phần, đồng thời chứng minh trọn vẹn kỹ năng phân tích, thiết kế cơ sở dữ liệu, lập trình logic và triển khai vận hành hệ thống web của sinh viên.

TÀI LIỆU THAM KHẢO

- 1) Trường Đại học Bình Dương, “Đề tiểu luận cuối kỳ - Học phần Chuyên đề 1”, Học kỳ 3, năm học 2025-2026.
- 2) Node.js Documentation, “About Node.js”, <https://nodejs.org/en/about> (truy cập tháng 07/2026).
- 3) Render Documentation, “Web Services”, <https://render.com/docs/web-services> (truy cập tháng 07/2026).
- 4) Vercel Documentation, “Deploying Git Repositories”, <https://vercel.com/docs/git> (truy cập tháng 07/2026).
- 5) Source code Hệ thống Quản lý khóa học trực tuyến, <https://github.com/22050025-coder/do-an-web.git>

